CS6886 – Systems Engineering for Deep Learning

Assignment 5

Name: Saptajit Banerjee
Roll No: ID25S001

1. Machine

Inputs:
All values come from machine_spec.txt.

Symbol	Input	Value
C	Physical cores	48, from 2 sockets of 24
f_max	Max single-core clock	4.0 GHz
f_base	Base clock, used for all cores	2.4 GHz
W	SIMD width for FP32	512 bits ÷ 32 bits = 16 floats
U	FMA units per core	2
F	FLOPs per FMA	2, one multiply and one add
S	Sockets	2
Ch	Memory channels per socket	6
R	Memory transfer rate	2933 MT/s, million transfers per second
B	Bytes per transfer per channel	8, for a 64-bit channel

Formulas:

- FLOPs per Cycle per Core = U × W × F
- Peak, One Core = (U × W × F) × f_max
- Peak, All Cores = C × (U × W × F) × f_base
- Memory Bandwidth = S × Ch × R × B
- Ridge Point = Peak ÷ Memory Bandwidth

Calculations:

- FLOPs per cycle per core = 2 × 16 × 2 = 64 FLOP/cycle
- Peak FP32, One Core = 64 FLOP/cycle × 4.0 × 10^9 cycles/s = 256 GFLOP/s
- Peak FP32, All Cores = 48 × 64 FLOP/cycle × 2.4 × 10^9 cycles/s = 7372.8 GFLOP/s
- Memory Bandwidth = 2 × 6 × 2933 × 10^6 transfers/s × 8 B = 281.6 GB/s
- Ridge Point, One Core = 256 GFLOP/s ÷ 281.6 GB/s = 0.91 FLOP/byte
- Ridge Point, All Cores = 7372.8 GFLOP/s ÷ 281.6 GB/s = 26.2 FLOP/byte

2. Naive Baseline

Formulas:

- FLOPs = $4 \times N^2 \times d$
- GFLOP/s = $4N^2d \div \text{time}$
- % of peak = GFLOP/s ÷ 256 × 100

Results:

Input (N, d)	FLOPs	Time (ms)	GFLOP/s	% of Peak
1024, 64	0.268×10^9	124.3	2.16	0.84%
2048, 64	1.074×10^9	500.6	2.14	0.84%
4096, 64	4.295×10^9	2258.1	1.90	0.74%
4096, 128	8.590×10^9	8133.7	1.06	0.41%

How time changes with N

Work grows with N^2. So doubling N should make the time 4× longer. From N = 1024 to N = 2048 the time grows 4.0×, which matches the expectation.

From N = 2048 to N = 4096 the time grows 4.5×, more than expected. At N = 4096 the S matrix occupies 64 MB and no longer fits in the 35.75 MB L3 cache, so it must be stored in main memory, which is much slower.

How time changes with d

Work grows linearly with d, so doubling d should double the time. At N = 4096, doubling d increases the time 3.6×. The dependency chains in step 1 become longer, and the strided reads of V in step 3 no longer fit in the L2 cache, as explained below.

Where the time goes

Each step was timed separately:

Input	Step 1: Scores	Step 2: Softmax	Step 3: Output
N=1024, d=64	37%	4%	59%
N=4096, d=64	33%	4%	64%
N=4096, d=128	22%	1%	77%

Almost all of the time is spent in the two matrix multiplications, with step 3 the most expensive. Softmax accounts for a small share.

Step 1 is slow because its operations execute one at a time. Each score is a sum:
acc = acc + Q[i][k] × K[j][k] for k = 0 … d-1

Each step needs the previous value of acc, so the additions form a chain. An FMA takes 4 cycles to finish, so the core does one useful FMA every 4 cycles and sits idle the rest of the time.

The compiler could split the sum across 16 SIMD lanes, but that changes the order of the additions. Floating-point addition gives slightly different results in a different order, so without the -ffast-math flag the compiler is not permitted to do this, and the loop remains scalar.

Step 3 is slow because it reads V in the wrong direction. The inner loop is:
acc = acc + S[i][j] × V[j][k] for j = 0 … N-1

V is stored row by row, but this loop walks down a column. Each step jumps to the next row, d floats away. Memory always arrives in 64-byte chunks of 16 floats, but the loop uses only 1 float from each chunk before jumping. Most of each loaded chunk is therefore wasted.

With d = 128 the jump is 512 bytes. Because of how the cache maps addresses, those jumps reuse only 1/8 of the L2 cache, so V no longer fits and has to come from the slower L3. This is why doubling d costs 3.6× instead of 2×. Step 3 also has the same one-at-a-time chain problem as step 1.

Softmax calls expf N^2 times but accounts for only 1 to 4% of the time. It becomes the main bottleneck once the matrix multiplications are optimised in Q3.

3. Optimisations (Single Thread)

Formulas:

- GFLOP/s = $4N^2d \div$ Time
- % of peak = GFLOP/s ÷ 256 × 100
- Speedup = Naive time ÷ Variant Time

(a) attn_loops.c

Changes

The naive inner loops had two problems. They added into one running sum, which forms a dependency chain, and step 3 read V down a column. The new version changes the loop order so that the innermost loop:

- Walks along a row of memory, one float after the next, and
- Updates many independent outputs instead of one sum.

For step 1, K is first transposed into K^T so that its rows are read in order. The loops become:
Step 1: S[i][j] += Q[i][k] × K^T[k][j] inner loop over j
Step 3: O[i][k] += S[i][j] × V[j][k] inner loop over k

Each iteration of the inner loop is now independent and reads the next float in memory. The compiler can therefore vectorise these loops, processing 8 floats per instruction.

Input (N, d)	Time (ms)	GFLOP/s	% of peak	Speedup
1024, 64	21.39	12.55	4.90%	5.8×
2048, 64	85.42	12.57	4.91%	5.9×
4096, 64	380.78	11.28	4.41%	5.9×
4096, 128	794.39	10.81	4.22%	10.2×

The d = 128 case improves most, because naive suffered most there from the column reads of V.

Limitations

- K^T and V are re-read for every row. Each output row reads the whole of K^T and V. With N = 4096 they are too big for L2, so they come from the slower L3. The CPU waits for data.
- Sums live in memory, not registers. Every FMA loads the running sum, updates it and stores it back. That is three memory operations for one FMA.
- Softmax is still scalar. It now takes about 28% of the time.

(b) attn_tiled.c

Changes

Instead of computing one output at a time, the code computes a small block of outputs together: 6 rows × 64 columns. That block of 384 values lives in 24 vector registers, the fastest storage on the CPU.
Inside the loop, each step:

- Loads 64 values of B once and reuses each of them for 6 rows,
- Loads 6 values of A once and reuses each of them for 64 columns, and
- Does 24 vector FMAs with almost no memory traffic.

This is register tiling: data is loaded once and reused many times from registers. On top of that, the loops are ordered so that the chunk of data being reused fits in the L1 or L2 cache. This is cache tiling. Both matrix multiplications use the same tiled kernel, written with AVX-512 instructions that handle 16 floats at once. S is still stored in full.

Input (N, d)	Time (ms)	GFLOP/s	% of peak	Speedup
1024, 64	9.58	28.01	10.94%	13.0×
2048, 64	38.28	28.05	10.96%	13.1×
4096, 64	160.42	26.77	10.46%	14.1×
4096, 128	189.28	45.38	17.73%	43.0×

Limitations

Softmax. The two matrix multiplications became about 6× faster, but softmax did not change. It still calls the scalar expf once per score, about 16.8 million times at N = 4096. At N = 4096, d = 64:

Step	Time
Both matrix multiplications	50 ms
Softmax	110 ms, 68% of the total

(c) attn_fused.c

Changes
Instead of finishing step 1 for all rows, then step 2, then step 3, the code handles one row completely before moving on:

```
for each row i:
   compute the N scores of row i      into a small buffer
   softmax that buffer
   O[i] = buffer × V
```

The buffer holds only N floats, 16 KB at N = 4096, so it stays in the L1 cache. The full N × N matrix S is never stored. The inner loops are the same as in the loops variant, so the only difference between the two is the fusion.

Input (N, d)	Time (ms)	GFLOP/s	% of peak	Speedup
1024, 64	21.13	12.71	4.96%	5.9×
2048, 64	94.45	11.37	4.44%	5.3×
4096, 64	452.96	9.48	3.70%	5.0×
4096, 128	776.44	11.06	4.32%	10.5×

It is not faster than loops and is slightly slower for large N:

- Writing S was never the bottleneck. Fusion saves the cost of writing S to memory and reading it back. In the loops variant that cost was only about 19 ms out of 381 ms.
- Each row now needs both K and V. At N = 4096, d = 64 together they are 2 MB, more than the 1 MB L2 cache, so both come from the slower L3 for every row. The loops variant only cycled through one of them at a time.

Limitations
Performance is limited by reading K and V from L3 for every row. Fusion only pays off when it is combined with tiling, so that each piece of K and V is reused by many rows. That is what part (d) does.

(d) attn_best.c: fastest single-threaded version

Changes
This combines everything that worked, in the style of FlashAttention.

1. Fusion plus tiling. The code takes 96 rows of Q at a time and walks through K and V in chunks of 64 keys. Each chunk of K and V loaded from memory is reused by all 96 rows, not just one. Scores are computed with the 6 × 64 register tile from part (b).

2. Online softmax. Softmax normally needs the whole row first, to find its maximum. Online softmax keeps a running maximum and a running sum per row, and corrects earlier results whenever a bigger maximum appears. So scores can be used immediately, chunk by chunk, and S never leaves the registers.

3. A fast vectorised exponential. expf is replaced with a custom version that computes 16 exponentials at once using a small polynomial. Its relative error is about 2×10^{-7}, comparable to standard single-precision accuracy.

4. Additional optimisations:
    - Row maxima are found entirely inside vector registers.
    - The scaling factor is folded into an existing FMA instruction.
    - V is copied to aligned memory, so loads do not straddle two cache lines.

Input (N, d)	Time (ms)	GFLOP/s	% of peak	Speedup
1024, 64	2.06	130.50	50.98%	60.4×
2048, 64	7.71	139.27	54.40%	64.9×
4096, 64	30.70	139.91	54.65%	73.6×
4096, 128	59.45	144.49	56.44%	136.8×

Why it works:

- It resolves the limitation of fused. Each piece of K and V is reused by 96 rows instead of 1, so memory is no longer the bottleneck.
- It resolves the limitation of tiled. Exponentials are computed 16 at a time at low cost.
- Data stays close to the core. Scores and probabilities never leave registers or the L1 cache.

Limitations

Performance is limited by the core itself, as the FMA units are busy almost all the time. The remaining gap has two causes:

- Extra vector work. The exponentials, maxima and rescaling also use the vector units. They take time but do not count as attention FLOPs.
- The CPU slows its own clock. It runs below 4.0 GHz when doing heavy AVX-512 work, which caps any program at about 70% of the 256 GFLOP/s figure on this machine.

4. OpenMP

Parallelisation:

The OpenMP version uses the same kernel as attn_best.c. The query rows are divided into blocks of up to 96 rows, sized so that each thread receives about two blocks. Each block writes only its own rows of O and keeps its own running maximum and sum for the online softmax, so the blocks are independent and no locks are required. The blocks are distributed with schedule(dynamic, 1), so a thread that finishes early takes the next available block. The packing of K and V is also parallelised, and each thread uses private accumulators. Threads are pinned one per core, filling one socket of 24 cores before using the second. Each reported time is the best of 7 runs.

Formulas:

- Speedup S = T1 ÷ Tp T1 = time with 1 thread, Tp = time with p threads
- Efficiency E = S ÷ p 1.00 indicates perfect scaling

Results:

N = 4096, d = 64

Threads p	Time (ms)	Speedup	Efficiency
1	30.85	1.0	1.00
2	16.10	1.9	0.96
4	8.42	3.7	0.92
8	5.11	6.0	0.76
12	3.57	8.6	0.72
16	2.75	11.2	0.70
24	2.16	14.3	0.60
32	2.05	15.1	0.47
40	1.71	18.1	0.45
48	1.67	18.4	0.38

N = 4096, d = 128

Threads p	Time (ms)	Speedup	Efficiency
1	58.67	1.0	1.00
2	30.37	1.9	0.97
4	16.04	3.7	0.91
8	9.64	6.1	0.76
12	6.87	8.5	0.71
16	5.23	11.2	0.70
24	4.40	13.3	0.56
32	3.86	15.2	0.47
40	3.47	16.9	0.42
48	23.52	2.5	0.05

N = 2048, d = 64

Threads p	Time (ms)	Speedup	Efficiency
1	8.10	1.0	1.00
2	4.15	2.0	0.98
4	2.38	3.4	0.85
8	1.34	6.1	0.76

12	0.92	8.8	0.73
16	0.73	11.2	0.70
24	0.59	13.7	0.57
32	0.64	12.7	0.40
40	0.59	13.8	0.35
48	0.56	14.6	0.30

N = 1024, d = 64

Threads p	Time (ms)	Speedup	Efficiency
1	2.14	1.0	1.00
2	1.16	1.8	0.92
4	0.64	3.4	0.84
8	0.37	5.9	0.73
12	0.30	7.1	0.59
16	0.24	8.8	0.55
24	0.21	10.3	0.43
32	0.28	7.7	0.24
40	0.28	7.5	0.19
48	0.30	7.2	0.15

The fastest run is N = 4096, d = 64 at 48 threads: 1.67 ms, which is 1350× faster than naive.

Scaling Explanation

Up to 4 threads the speedup is almost perfect, with efficiency of at least 0.84. Beyond that, efficiency falls steadily, for four reasons.

1. The CPU slows down when more cores are busy. Heavy AVX-512 work draws a lot of power. To stay within its power limit, the chip lowers the clock as more cores become active. Each extra core therefore adds less than the one before.

   This is visible even without attention code. A program that does nothing but FMAs gets only 15.1× faster on 24 cores and 24.7× on 48 cores. That is the best any compute-heavy code can do here. At 24 threads the attention code reaches 14.3×, 95% of that limit. So the code itself scales well, and the drop in efficiency comes mostly from the falling clock.

2. Small inputs run out of work. N = 1024 takes only 0.2 ms with 24 threads. At that scale, fixed costs become a big share of the time:

   - Starting and stopping the threads,

- The packing step and the wait for all threads to finish it, and
- The last blocks, which do not divide evenly across threads

Adding more threads makes each thread's share even smaller while these costs stay the same. That is why N = 1024 gets slower beyond 24 threads, and N = 2048 stops improving. The larger N = 4096 inputs have enough work to keep improving up to 40 threads.

3. Beyond 24 threads, the second chip joins in. Threads 25 to 48 run on the second chip. Q, K and V sit in the first chip's memory, so the second chip has to fetch them across the link between chips, which is slower. Synchronising threads across two chips also takes longer. This is why the speedup curve flattens after 24 threads.

4. Other users' processes interfere at 48 threads. The measurements were taken on a shared login node. At 48 threads every core is in use, including two cores occupied by another user's processes running at 100% CPU. The operating system periodically pauses one of the OpenMP threads to run those processes, and all threads must wait for the slowest one at the end of the parallel region.

   Timing of individual blocks confirmed this: every block took its normal 0.7 to 1.6 ms, but one thread finished about 10 ms after the others. This explains the 23.52 ms result for N = 4096, d = 128 at 48 threads, whereas the same input takes 3.47 ms at 40 threads.

5. Roofline

Roofline Construction
The roofline uses the theoretical values from Q1:

Parameter	Value
Peak FP32, one core	256 GFLOP/s
Peak FP32, all cores	7372.8 GFLOP/s
Memory bandwidth	281.6 GB/s
Ridge point, one core	0.91 FLOP/byte
Ridge point, all cores	26.2 FLOP/byte

The attainable performance at arithmetic intensity AI is:
Attainable GFLOP/s = min(Peak , AI × Memory bandwidth)

Two rooflines are drawn: one for a single core, which applies to the single-threaded variants, and one for all 48 cores, which applies to the OpenMP variant.

Computation of Arithmetic Intensity

Arithmetic intensity is the number of floating-point operations divided by the number of bytes transferred to and from main memory:

$AI = FLOPs \div Bytes$

The FLOP count is the same for every variant:
$FLOPs = 4N^2d = 4 \times 4096^2 \times 64 = 4.295 \times 10^9$

The byte count depends on whether the N × N matrix S is stored in main memory. All data is FP32, 4 bytes per value.

naive, loops, tiled. These variants read Q, K and V and write O, which is 4Nd values. They also store S, and S passes through main memory four times: written in step 1, read and written by softmax, and read in step 3. That adds $4N^2$ values.

- $Bytes = 4 \times (4Nd + 4N^2) = 16Nd + 16N^2$
- $AI = 4N^2d \div (16Nd + 16N^2) = 4.295 \times 10^9 \div 272.6 \times 10^6 = 15.8$ FLOP / byte

fused, best, omp. S is never stored in main memory, so only Q, K, V and O are transferred.

- $Bytes = 4 \times 4Nd = 16Nd$
- $AI = 4N^2d \div 16Nd = N \div 4 = 1024$ FLOP / byte

Position of each variant at N = 4096, d = 64

Variant	Threads	AI (FLOP/byte)	Achieved (GFLOP/s)	Roofline limit (GFLOP/s)	% of roofline limit
naive	1	15.8	1.90	256	0.74%
loops	1	15.8	11.28	256	4.41%
tiled	1	15.8	26.77	256	10.46%
fused	1	1024	9.48	256	3.70%
best	1	1024	139.91	256	54.65%
omp	48	1024	2567.2	7372.8	34.82%

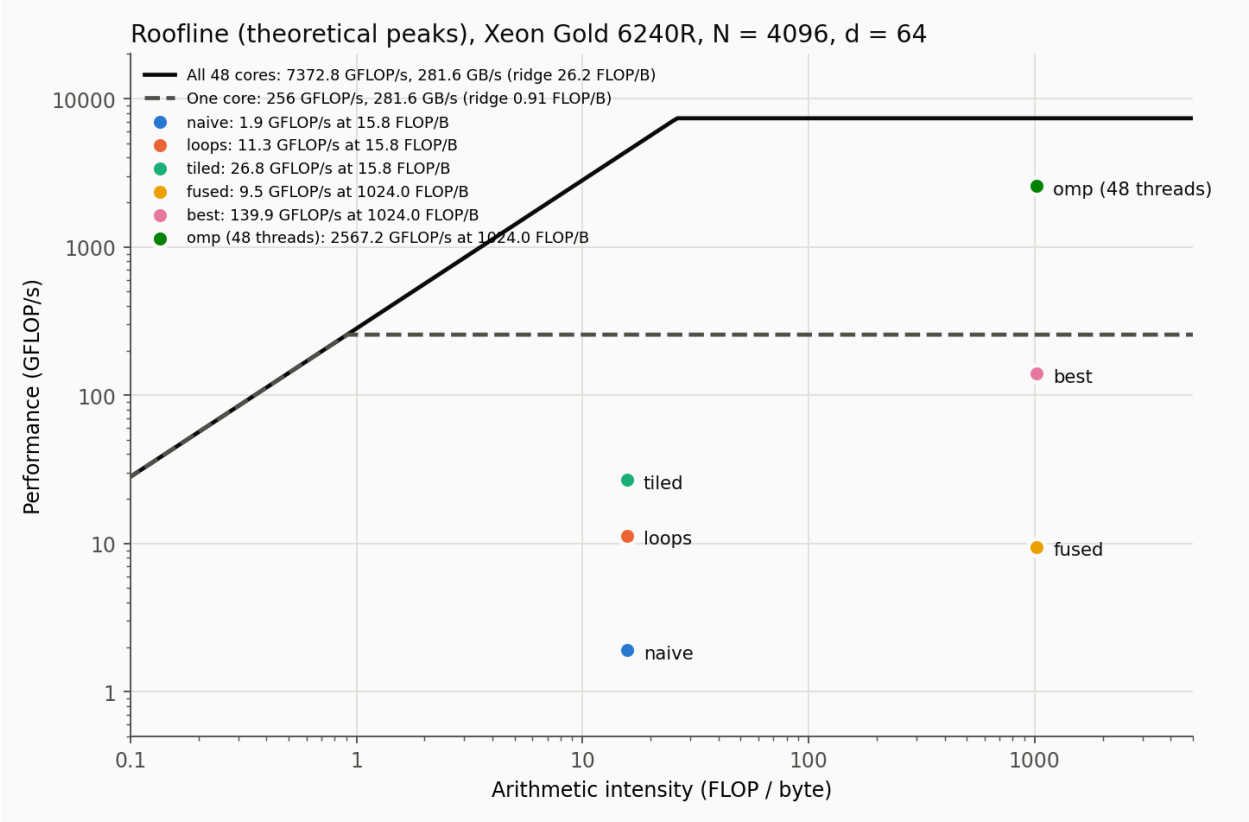
Every variant lies to the right of its ridge point: 0.91 FLOP/byte for one core and 26.2 FLOP/byte for all cores. By the roofline, main-memory bandwidth limits none of them. Each one is bounded only by the compute peak. The gap between each point and the roof is explained below.

Limiting Factor of Each Variant

- **naive:** The inner loops are not vectorised. Each addition depends on the previous one, so every FMA waits for the last to finish. Step 3 also reads V column-wise, which causes frequent cache misses. Performance is limited by instruction latency and cache misses, not by compute peak or memory bandwidth.

- **loops:** The inner loops are vectorised, but every output row reloads all of K^T and V from the L3 cache. Relative to L3 traffic, the intensity is only 0.5 FLOP/byte, so performance is limited by L3 cache bandwidth. The running sums are also kept in memory rather than in registers.
- **tiled:** Register and cache tiling make the matrix multiplications fast. Softmax, with its scalar exponential function, now takes 68% of the run time. Those exponentials are not counted as FLOPs, so the point sits low on the plot. Performance is limited by the scalar softmax.
- **fused:** Storing S is avoided, which moves the point from 15.8 to 1024 FLOP/byte. K and V are still reloaded from L3 for every row, at 0.5 FLOP/byte relative to L3. The point therefore moves right without moving up. Performance is limited by L3 cache bandwidth, as in loops.
- **best:** Each block of K and V loaded into cache is reused by 96 query rows, which raises the intensity relative to L3 to 48 FLOP/byte. The exponential is vectorised, and S stays in registers. Performance is limited by compute.

  The remaining gap to 256 GFLOP/s has two causes. First, the exponentials and other softmax operations share the same vector units. Second, the processor runs below its 4.0 GHz maximum clock during heavy AVX-512 work.
- **omp:** The data flow is the same as best, so the variant is compute-bound. The gap to 7372.8 GFLOP/s has three causes:
  - The clock drops as more cores become active,
  - There is extra cost when threads run on both sockets, and
  - Other users' processes on the shared machine delay individual threads.


Roofline (theoretical peaks), Xeon Gold 6240R, N = 4096, d = 64
All 48 cores: 7372.8 GFLOP/s, 281.6 GB/s (ridge 26.2 FLOP/B)
One core: 256 GFLOP/s, 281.6 GB/s (ridge 0.91 FLOP/B)
naive: 1.9 GFLOP/s at 15.8 FLOP/B
loops: 11.3 GFLOP/s at 15.8 FLOP/B
tiled: 26.8 GFLOP/s at 15.8 FLOP/B
fused: 9.5 GFLOP/s at 1024.0 FLOP/B
best: 139.9 GFLOP/s at 1024.0 FLOP/B
omp (48 threads): 2567.2 GFLOP/s at 1024.0 FLOP/B
omp (48 threads)
best
tiled
loops
fused
naive
Performance (GFLOP/s)
10000
1000
100
10
1
0.1
1
10
100
1000
Arithmetic intensity (FLOP / byte)